

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ
ΤΜΗΜΑ ΜΗΧ Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ
ΤΟΜΕΑΣ ΥΛΙΚΟΥ ΚΑΙ ΑΡΧΙΤΕΚΤΟΝΙΚΗΣ ΥΠΟΛΟΓΙΣΤΩΝ
Εργαστήριο Ηλεκτρονικών & Λογικού Σχεδιασμού

ΕΡΓΑΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ
ΛΟΓΙΚΟΥ ΣΧΕΔΙΑΣΜΟΥ

Γ.Φ. ΑΛΕΞΙΟΥ - Ε.Ζ. ΨΑΡΑΚΗΣ

Πάτρα 2018

Ακαδ. Έτος : 2020-21

Εαρινό Εξάμηνο : 2020-21

Ονοματεπώνυμο: ΚΙΟΥΡΤ- ΜΕΧΜΕΤΑΛΗ ΑΧΜΕΤ

Α.Μ. : 1084672

Ονομ/νυμο Συνεργάτη: _____

Α.Μ. : _____

Ονομ/νυμο Συνεργάτη: _____

Α.Μ. : _____

Τμήμα : _____

Άσκηση: __ ΑΣΚΗΣΗ 6 __

Αναλυτική Βαθμολογία Πρακτικού Μέρους:

	Βαθμός Αριθ.	Αξιολογητή Ολογραφ.	Υπογραφή Αξιολογητή
Άσκηση 1			
Άσκηση 3			
Άσκηση 5			
Άσκηση 6			
Άσκηση 7			
Μ.Ο.			

ΑΣΚΗΣΗ 6**ΗΜΕΡΟΜΗΝΙΑ:****ΘΕΜΑ: ΚΑΤΑΧΩΡΗΤΕΣ - REGISTERS**

(M. Mano, Γ' Έκδοση, Κεφ. 6, σελ. 301 - 315)

ΘΕΩΡΙΑ.

Ένας καταχωρητής (register) είναι μια ομάδα από δυαδικά κύτταρα αποθήκευσης (flip-flops) τα οποία όπως ξέρουμε είναι κατάλληλα για την αποθήκευση δυαδικών πληροφοριών. Ένας καταχωρητής των "n" bits περιέχει "n" flip-flops και επομένως μπορεί να αποθηκεύσει κάθε πληροφορία των "n" bits. Με την ευρύτερη έννοια, ένας καταχωρητής αποτελείται από ένα σύνολο flip-flops τα οποία όπως είπαμε κρατούν τις δυαδικές πληροφορίες και από ένα σύνολο πυλών για την επίτευξη της μεταφοράς των πληροφοριών.

Η μεταφορά νέων πληροφοριών μέσα σ' ένα καταχωρητή λέγεται "φόρτωση" του καταχωρητή. Ανάλογα με τον τρόπο μεταφοράς των δεδομένων στον καταχωρητή διακρίνουμε δύο τύπους καταχωρητών:

1. Τους καταχωρητές παράλληλης φόρτωσης (parallel load registers).
2. Τους καταχωρητές ολίσθησης (shift registers).

Στον τύπο (1) των καταχωρητών όλα τα bits του καταχωρητή φορτώνονται την ίδια χρονική στιγμή, δηλαδή η φόρτωση γίνεται παράλληλα. Στον τύπο (2) των καταχωρητών η πληροφορία διαδίδεται κατά μήκος του καταχωρητή. Ένας τέτοιος καταχωρητής αποτελείται από μία αλυσίδα από flip-flop συνδεδεμένα στη σειρά, με την έξοδο του ενός να τροφοδοτεί την είσοδο του γειτονικού του. Όλα τα flip-flops παίρνουν ένα κοινό ρολόι και το οποίο προκαλεί την ολίσθηση από την μία βαθμίδα στην επόμενη. Σ' ένα τέτοιο καταχωρητή η είσοδος της πρώτης βαθμίδας ονομάζεται "σειριακή είσοδος" (serial input) ενώ η έξοδος της τελευταίας βαθμίδας, ονομάζεται "σειριακή έξοδος" (serial output).

ΠΡΟΣΟΧΗ : Πριν ξεκινήσετε την υλοποίηση της άσκησης κατεβάστε **ΟΠΩΣΔΗΠΟΤΕ** από το eclass του εργαστηρίου (από την κατηγορία “Έγγραφα\Logisim-Evolution”) το .zip αρχείο με την ανανεωμένη βιβλιοθήκη για το Logisim (**CEID_LogicDesign_LogisimEV_library_v1.1**). Για το πως εισάγουμε βιβλιοθήκη στο Logisim-Ev, συμβουλευτείτε την παρουσίαση για τον simulator που βρίσκεται στα “Έγγραφα/Logisim-Evolution” του eclass του εργαστηρίου.

ΜΑΖΙ με την αναφορά **ΠΡΕΠΕΙ ΟΠΩΣΔΗΠΟΤΕ** να παραδώστε και το αρχείο .circ του simulator

ΔΙΑΔΙΚΑΣΙΑ

1. Υλοποιήστε στον simulator έναν καταχωρητή ολίσθησης (shift-register) των 4-bit με D flip-flops, βεβαιωθείτε, με την χρήση παλμογράφου, για την σωστή λειτουργία του κυκλώματός σας και δώστε τα απαιτούμενα screenshots.

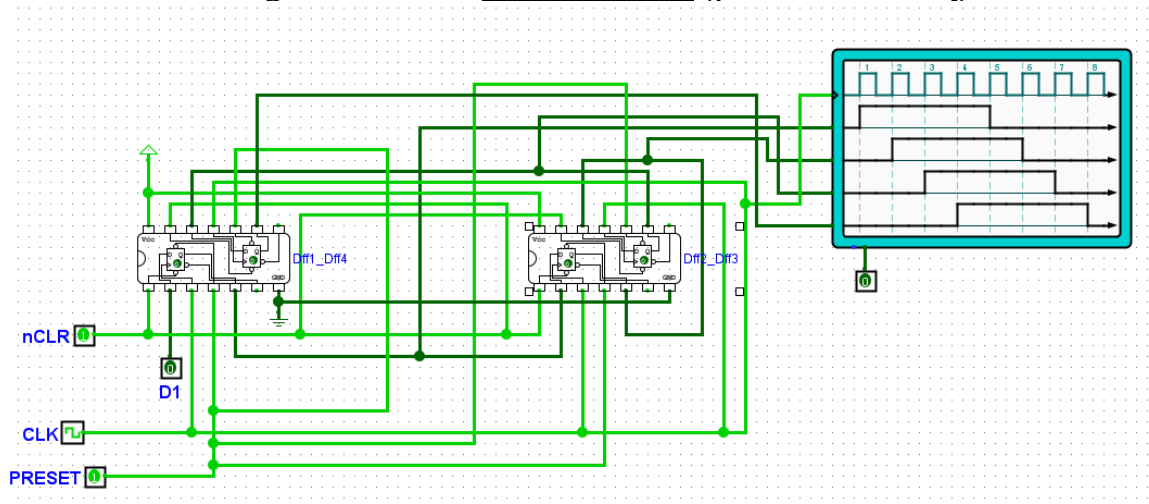
Παρατηρήσεις:

- Χρησιμοποιείτε **ΜΟΝΟ το ολοκληρωμένο 7474** από την βιβλιοθήκη **TTL** και τον παλμογράφο (*Digital oscilloscope*) από την βιβλιοθήκη **Input/Output-Extra**. Ρυθμίστε τον παλμογράφο να απεικονίζει **5 σήματα συνολικά**: το clock (με συχνότητα 1Hz) και τις εξόδους των flip-flop Q0-Q3 **για 8 καταστάσεις**. Μπορείτε επίσης να χρησιμοποιήσετε και leds για οπτική απεικόνιση των εξόδων των flip-flops
- **ΠΡΟΣΟΧΗ**: Οι είσοδοι CLEAR(ή *Reset, R, nCLR*) και PRESET(ή *Set, S, nPRE*) στο 7474 είναι **Active Low**!
- **ΠΡΟΣΟΧΗ**: Για να ξεκινήσει το Flip-Flop να λειτουργεί σωστά πρέπει πρώτα να θέσετε την είσοδο CLEAR(ή *Reset, R, nCLR*) στο “0” (**CLEAR=0, εκκαθάριση**) και έπειτα να τη θέσετε στο λογικό “1”. Η είσοδος PRESET(*Set, S, nPRE*) πρέπει να συνδεθεί εξαρχής στο λογικό “1”
- Χρησιμοποιήστε ένα pin εισόδου για να εισάγεται στην είσοδο του καταχωρητή το bit που θα “ολισθαίνει”. Το bit αυτό θα έχει **ΣΥΝΕΧΩΣ** την τιμή “1”.
- Επιβεβαιώστε πριν την έναρξη της εξομοίωσης ότι η συχνότητα λειτουργίας (tick frequency) του CLOCK είναι ίση με 1Hz. (Συμβουλευτείτε την παρουσίαση για τον simulator που βρίσκεται στα “Εγγγραφα” του eclass του εργαστηρίου)

ΚΥΚΛΩΜΑ**Σύντομη επεξήγηση της επιλογής σχεδίασης**

Για την σχεδίαση του συγκεκριμένου κυκλώματος παίρνουμε ως παράδειγμα το κλασικό shift register που συνδέουμε κάθε Q του προηγούμενου flip flop στο επόμενο Data Input του flip flop. Επίσης, συνδέουμε το πρώτο Data Input σε ένα pin για να δίνουμε της χρειαζόμενες τιμές για τύπωση των 8 καταστάσεων στο παλμογράφο μας. Που όταν κάνουμε το D=1 παίρνουμε 1 σε κάθε επόμενη κατάσταση. Ενώ όταν κάνουμε το D=0 βλέπουμε ότι αυτή την φορά παίρνουμε 0 σε κάθε επόμενη κατάσταση.

- Screenshot shift-register των 4-bit με παλμογράφο (για 8 καταστάσεις)



2. Σκοπός του ερωτήματος αυτού είναι να σχεδιάσετε έναν καταχωρητή ολίσθησης των 4-bit «δυναμικής λειτουργίας». Δηλαδή θα πρέπει δηλαδή ο καταχωρητής να δημιουργεί έναν "1" ο οποίος θα διαδίδεται σειριακά από το κάθε flip-flop στο γειτονικό του με κυκλικό τρόπο. Για τον σκοπό τροποποιήστε το κύκλωμα του ερωτήματος 6.1 συνδέοντας την "σειριακή έξοδο" του στην "σειριακή" του είσοδο" και **τροποποιώντας την διασύνδεση των PRESET και CLEAR της τελευταίας βαθμίδα του**. Εξηγείστε σύντομα την λειτουργία του κυκλώματος και δώστε τα απαραίτητα screenshots. Που έχουν εφαρμογή τέτοιου είδους κυκλώματα;

Παρατηρήσεις:

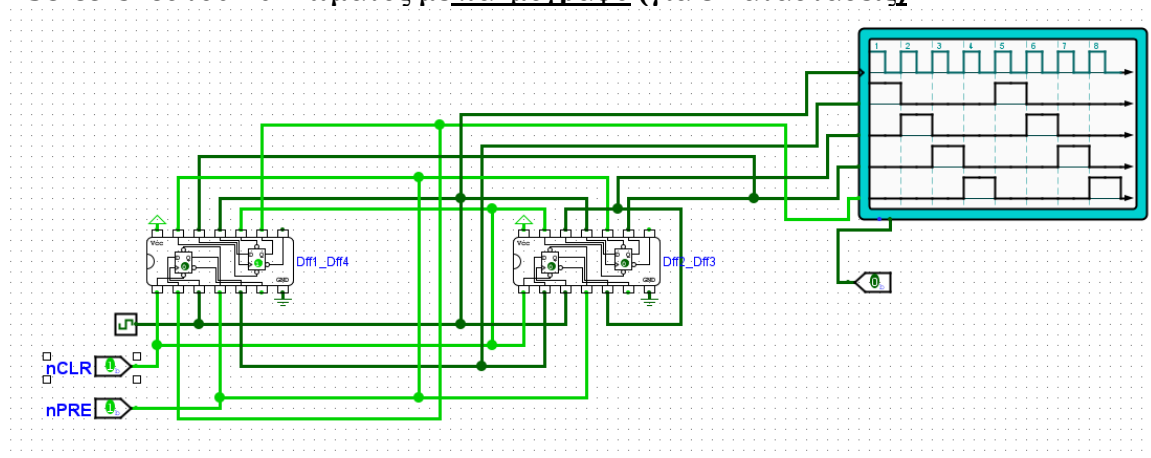
- Χρησιμοποιείτε **ΜΟΝΟ το ολοκληρωμένο 7474** από την βιβλιοθήκη **TTL** και τον παλμογράφο (*Digital oscilloscope*) από την βιβλιοθήκη **Input/Output-Extra**. Ρυθμίστε τον παλμογράφο να απεικονίζει **5 σήματα συνολικά**: το clock (με συχνότητα 1Hz) και τις εξόδους των flip-flop Q0-Q3 **για 8 καταστάσεις**. Μπορείτε επίσης να χρησιμοποιήσετε και leds για οπτική απεικόνιση των εξόδων των flip-flops
- **ΠΡΟΣΟΧΗ:** Οι είσοδοι CLEAR(ή *Reset, R, nCLR*) και PRESET(ή *Set, S, nPRE*) στο 7474 είναι **Active Low!**
- **ΠΡΟΣΟΧΗ:** Για να ξεκινήσει το Flip-Flop να λειτουργεί σωστά πρέπει πρώτα να θέσετε την είσοδο CLEAR(ή *Reset, R, nCLR*) στο "0" (**CLEAR=0, εκκαθάριση**) και έπειτα να τη θέσετε στο λογικό "1". Η είσοδος PRESET(*Set, S, nPRE*) πρέπει να συνδεθεί εξαρχής στο λογικό "1"
- Επιβεβαιώστε πριν την έναρξη της εξομοίωσης ότι η συχνότητα λειτουργίας (tick frequency) του CLOCK είναι ίση με 1Hz. (Συμβουλευτείτε την παρουσίαση για τον simulator που βρίσκεται στα "Έγγραφα" του eclass του εργαστηρίου)

ΣΥΝΔΙΑΣΤΙΚΟ ΚΥΚΛΩΜΑ

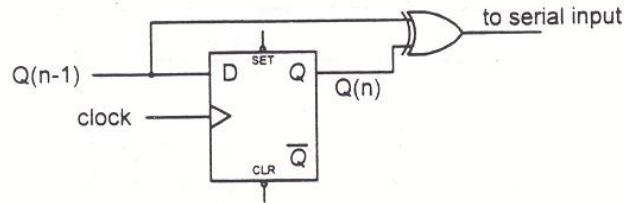
Σύντομη δικαιολόγηση της επιλογής σχεδίασης και της λειτουργίας του κυκλώματος

Για να έχουμε ένα shift register με δυναμική λειτουργία, αυτή την φορά δεν έχουμε ένα pin για την Data Input του πρώτου flip flop αλλά συνδέουμε το πρώτο D στο τελευταίο Q ώστε να διαδίδεται το σήμα και να γυρίσει κυκλικά ξανά στο πρώτο σήμα που μπορούμε να το παρατηρήσουμε στο παλμογράφο που χρησιμοποιούμε. Τέτοια λειτουργία έχει εφαρμογή σε κυκλώματα υψηλής ταχύτητας. Που συνίσταται στην καταστολή της δράσης του ρολογιού αντί για τον έλεγχο του σήματος μέσω πύλης.

- Screenshot του κυκλώματος με παλμογράφο (για 8 καταστάσεις)



3. Τροποποιήστε την τελευταία βαθμίδα του κυκλώματος του ερωτήματος 6.2 όπως φαίνεται στο Σχήμα 1. Καταγράψτε τις εξόδους Q0 – Q3 για τουλάχιστον 20 καταστάσεις και εξηγήστε την λειτουργία του κυκλώματος.



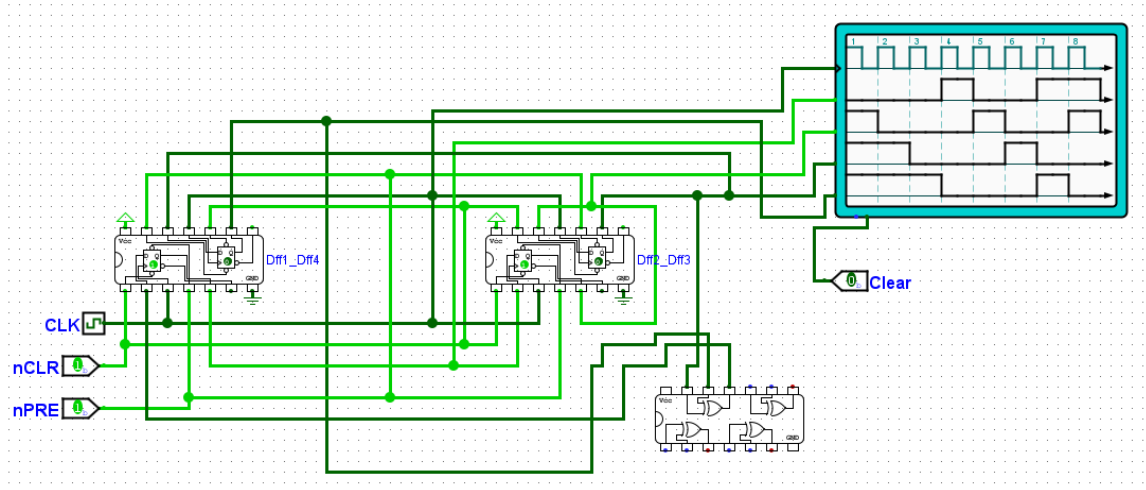
Σχήμα 1. Μετατροπή τελευταίας βαθμίδας καταχωρητή ολίσθησης

Παρατηρήσεις:

- Χρησιμοποιείτε **MONO** τα ολοκληρωμένα **7474 και 7486** από την βιβλιοθήκη **TTL** καθώς και τον παλμογράφο (*Digital oscilloscope*) από την βιβλιοθήκη **Input/Output-Extra**. Ρυθμίστε τον παλμογράφο να απεικονίζει **5 σήματα συνολικά**: Το clock (με συχνότητα **0,5Hz**) και τις εξόδους των flip-flop Q0-Q3 για **25 καταστάσεις**. Μπορείτε επίσης να χρησιμοποιήσετε και leds για οπτική απεικόνιση των εξόδων των flip-flops
- ΠΡΟΣΟΧΗ**: Οι είσοδοι CLEAR(ή *Reset, R, nCLR*) και PRESET(ή *Set, S, nPRE*) στο 7474 είναι **Active Low**!
- ΠΡΟΣΟΧΗ**: Για να ξεκινήσει το Flip-Flop να λειτουργεί σωστά πρέπει πρώτα να θέσετε την είσοδο CLEAR(ή *Reset, R, nCLR*) στο “0” (**CLEAR=0, εκκαθάριση**) και έπειτα να τη θέσετε στο λογικό “1”. Η είσοδος PRESET(*Set, S, nPRE*) πρέπει να συνδεθεί εξαρχής στο λογικό “1”
- Επιβεβαιώστε πριν την έναρξη της εξομοίωσης ότι η συχνότητα λειτουργίας (tick frequency) του CLOCK **είναι ίση με 0,5Hz** (*σμβουλευτείτε την παρουσίαση για τον simulator που βρίσκεται στα “Εγγγραφα” του eclass του εργαστηρίου*), ώστε να προλαβαίνετε να καταγράφετε τις εξόδους των flip-flops.

ΚΥΚΛΩΜΑ

- Screenshot του κυκλώματος με παλμογράφο (για 8 καταστάσεις)



Έξοδοι των flip-flops

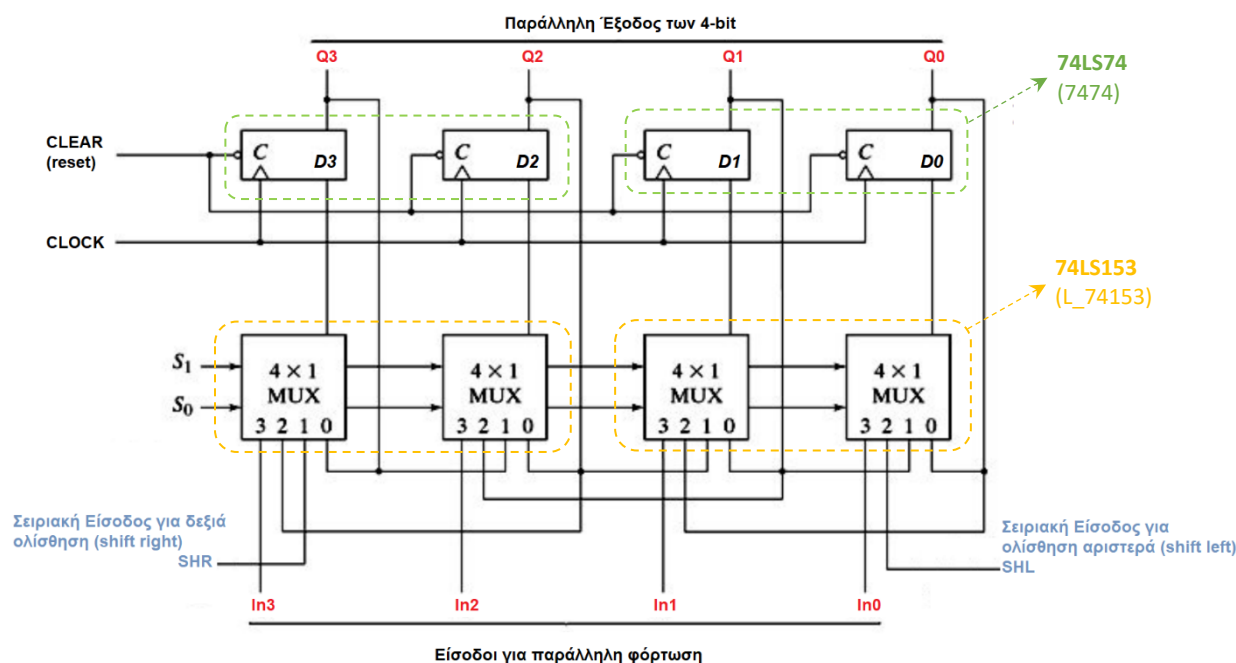
#Clocks	Q0	Q1	Q2	Q3
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	1	0	0	1
5	1	1	0	0
6	0	1	1	0
7	1	0	1	1
8	0	1	0	1
9	1	0	1	0
10	1	1	0	1
11	1	1	1	0
12	1	1	1	1
13	0	1	1	1
14	0	0	1	1
15	0	0	0	1
16	1	0	0	0
17	0	1	0	0
18	0	0	1	0
19	1	0	0	1
20	1	1	0	0
21	0	1	1	0
22	1	0	1	1
23	0	1	0	1
24	1	0	1	0
25	1	1	0	1

Σύντομη επεξήγηση της λειτουργίας του κυκλώματος

Το κύκλωμα που έχουμε φτιάξει για την άσκηση 3 είναι σχεδόν το κύκλωμα της δεύτερης άσκησης. Η μόνη διαφορά που υπάρχει είναι ότι αυτή την φορά συνδέουμε το Data Input του πρώτου Dff στην έξοδο του XOR, που στην είσοδο του παίρνει τα Q του Dff3 και Dff4. Που μπορούμε να παρατηρήσουμε από τον παλμογράφο μας ότι διαδίδεται το σήμα σε κάθε flip flop.

4. Το κύκλωμα του καταχωρητή του Σχήματος 2 εκτελεί, σύμφωνα με τις τιμές των σημάτων επιλογής S_1 , S_0 , τις λειτουργίες που περιγράφονται στον παρακάτω πίνακα. Αφού μελετήσετε και κατανοήσετε τον τρόπο λειτουργίας του, να υλοποιήσετε στον simulator το κύκλωμα αυτό.

S_1	S_0	Λειτουργία του Καταχωρητή
0	0	Διατηρεί την παρούσα κατάσταση
0	1	Δεξιά Ολίσθηση
1	0	Αριστερή Ολίσθηση
1	1	Παράλληλη Φόρτωση



Σχήμα 2. Κύκλωμα καταχωρητή παράλληλης φόρτωσης/σειριακής ολίσθησης

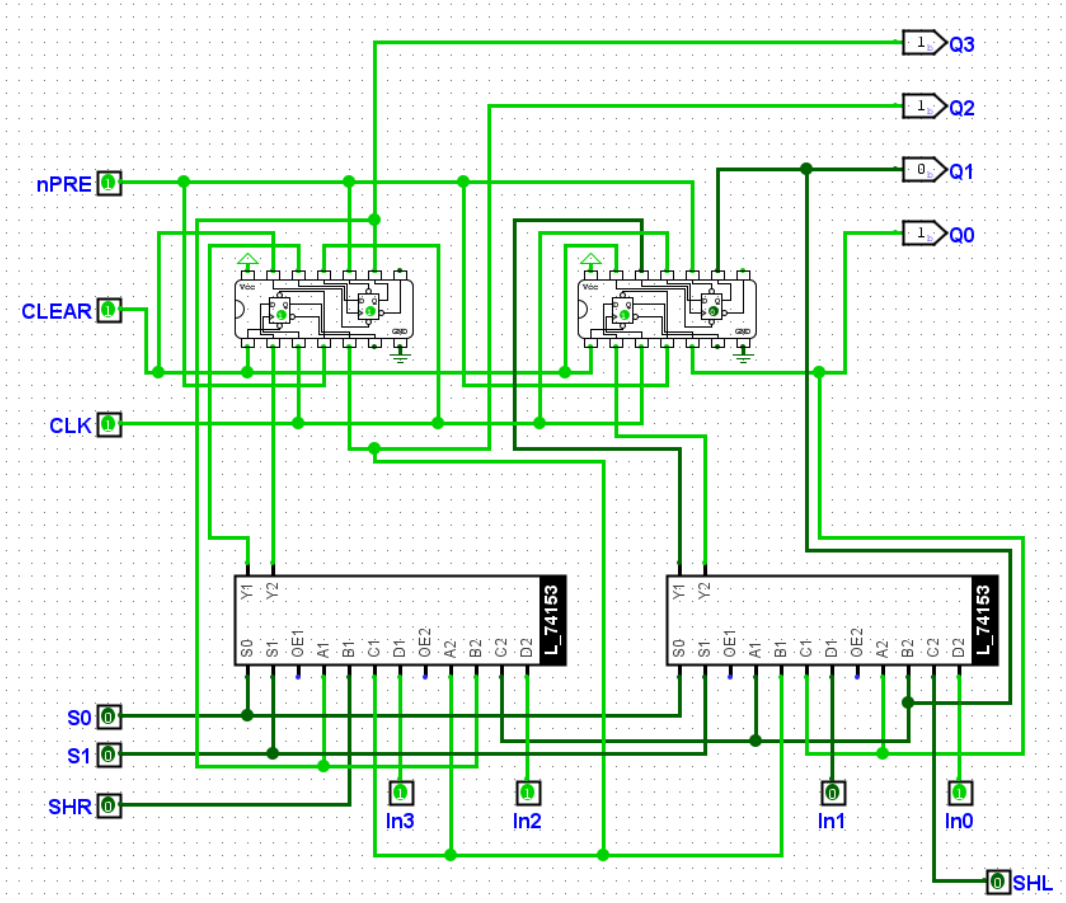
Παρατηρήσεις:

- Από την βιβλιοθήκη **CEID_LogiDesign_LogisimEV** χρησιμοποιείτε **MONO** ολοκληρωμένο **L_74153**. Από την βιβλιοθήκη **TTL MONO** το ολοκληρωμένο **7474**.
- ΠΡΟΣΟΧΗ:** Οι είσοδοι **CLEAR**(ή *Reset*, *R*, *nCLR*) και **PRESET**(ή *Set*, *S*, *nPRE*) στο 7474 είναι **Active Low!** Για να ξεκινήσει το Flip-Flop να λειτουργεί σωστά πρέπει πρώτα να θέσετε την είσοδο **CLEAR**(ή *Reset*, *R*, *nCLR*) στο “0” (**CLEAR=0, εκκαθάριση**) και έπειτα να τη θέσετε στο λογικό “1”. Η είσοδος **PRESET**(*Set*, *S*, *nPRE*) πρέπει να συνδεθεί εξαρχής στο λογικό “1”
- ΠΡΟΣΟΧΗ:** Οι είσοδοι **Enable** (oE1, oE2) του 74153 είναι **Active Low!**

- Χρησιμοποιείτε pins εξόδου για τις εξόδους Q3-Q0 του καταχωρητή γιατι **θα τον χρησιμοποιήσετε ως υποκύκλωμα στο επόμενο ερώτημα**
- Επιβεβαιώστε πριν την έναρξη της εξομοίωσης ότι η συχνότητα λειτουργίας (tick frequency) του CLOCK είναι ίση με **1Hz**. (Συμβουλευτείτε την παρουσίαση για τον simulator που βρίσκεται στα “Εγγραφα” του eclass του εργαστηρίου)
- Σε περίπτωση σχεδίασης/χρήσης υποκυκλώματος **πρέπει** να συμπεριλάβετε και ξεχωριστό screenshot για το υποκύκλωμα που χρησιμοποιείτε!!

ΚΥΚΛΩΜΑ

- Screenshot του κυκλώματος καταχωρητή του Σχήματος 2 για **S0=1, S1=1, In3-In0=1101**



Σύντομη επεξήγηση της λειτουργίας του κυκλώματος

Η λειτουργία του συγκεκριμένου κυκλώματος είναι σύμφωνα με το παραπάνω Σχήμα 2. Σύμφωνα με το κύκλωμα που παίρνουμε ως παράδειγμα δημιουργούμε ένα κύκλωμα που έχει 4 καταστάσεις λειτουργίας. Αρχικά στην κατάσταση $S_1 = 0$ και $S_0 = 0$ μπορούμε να παρατηρήσουμε ότι η κατάσταση παραμένει ίδια. Για την κατάσταση $S_{1,0} = 1$ βλέπουμε ότι γίνεται παράλληλη φόρτωση που δεν αλλάζει και μεταδίδονται όλες οι τιμές που έχουμε δηλώσει στα έξοδοι μας. Στην αριστερή ολίσθηση τα bits μπαίνουν από το σημείο SHR που τα bits μεταδίδονται προς αριστερά και η έξοδος Q_3 εξαφανίζεται. Επιπλέον, για την δεξιά ολίσθηση τα bits μετακινούν προς τα δεξιά και το αριστερότερο bit εξαφανίζεται. Για να έχουμε όλες αυτές τις αναφερόμενες καταστάσεις χρειάζεται να έχουμε την σύνδεση των SHR,SHL και τα είσοδοι για παράλληλη φόρτωση.

5. Χρησιμοποιώντας το κύκλωμα του Ερωτήματος 6.4 (ως υποκύκλωμα) να υλοποιήσετε στον simulator ένα κύκλωμα που να εκτελεί σειριακή πρόσθεση. Θα πρέπει να φορτώνετε παράλληλα 2 αριθμούς των 4-bits σε 2 καταχωρητές (του Ερωτ. 6.4), και έπειτα να τους ρυθμίσετε να εκτελέσουν δεξιά ολίσθηση. Κάθε ζεύγος bits σειριακής εξόδου των καταχωρητών πρέπει να προστίθεται σε έναν πλήρη αθροιστή και το κρατούμενο να αποθηκεύεται σε ένα D flip-flop, ώστε προστεθεί μαζί με το αμέσως επόμενο σημαντικό ζεύγος bits που προκύπτουν από τις σειριακές εξόδους των καταχωρητών. Η διαδικασία της πρόσθεσης πρέπει να ολοκληρώνεται μετά από **4 παλμούς ρολογιού (clocks) και το αποτέλεσμα της πράξεως να αποθηκεύεται στον πάλι καταχωρητή A.**

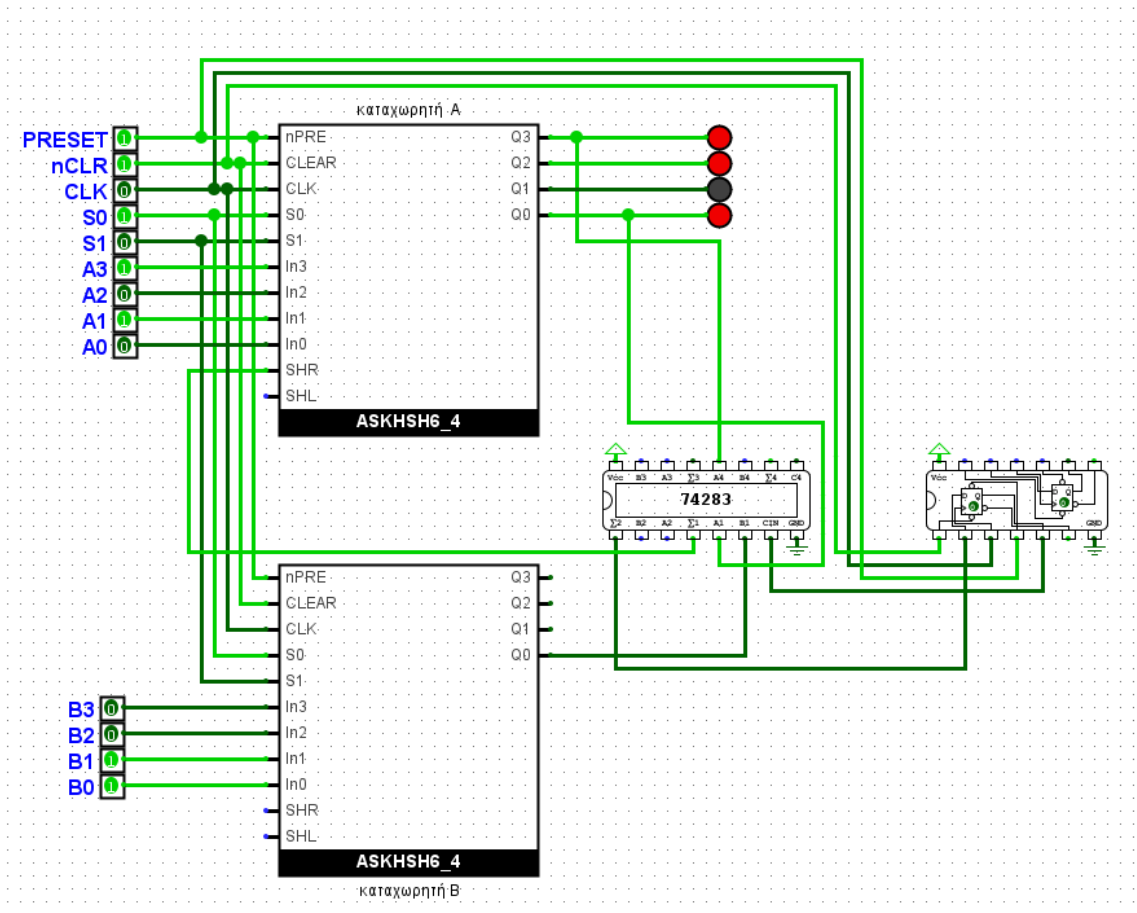
Παρατηρήσεις:

- Από την βιβλιοθήκη **CEID_LogicDesign_LogisimEV** χρησιμοποιείτε **MONO** το ολοκληρωμένο **L_74153**. Από την βιβλιοθήκη **TTL MONO** τα ολοκληρωμένα **7474** και **74283**. Χρησιμοποιείτε το 74283 ως αθροιστή του 1-bit (τα pins **A1, B1, S1, S2**).
- **ΠΡΟΣΟΧΗ:** Οι είσοδοι **CLEAR**(ή *Reset, R, nCLR*) και **PRESET**(ή *Set, S, nPRE*) στο 7474 είναι **Active Low!** Για να ξεκινήσει το Flip-Flop να λειτουργεί σωστά πρέπει πρώτα να θέσετε την είσοδο **CLEAR**(ή *Reset, R, nCLR*) στο “0” (**CLEAR=0, εκκαθάριση**) και έπειτα να τη θέσετε στο λογικό “1”. Η είσοδος **PRESET**(*Set, S, nPRE*) πρέπει να συνδεθεί εξαρχής στο λογικό “1”
- Για να ελέγξετε τη σωστή λειτουργία του κυκλώματος πιο εύκολα (αλλαγή τιμών των σημάτων Select, κτλ), προτείνεται να χρησιμοποιήσετε α) ένα απλό στοιχείο *Pin* (της βιβλιοθήκης *Wiring*), όπου θα του αναθέτετε εσείς τιμές “0” - “1”, προσομοιώνοντας έτσι τους παλμούς ρολογιού (αντί για το στοιχείο *Clock*) και β) *leds* στις εξόδους του καταχωρητή A. Αν όμως επιλέξετε την χρήση του στοιχείου *Clock*, επιβεβαιώστε πριν την έναρξη της εξομοίωσης ότι η συχνότητα λειτουργίας (tick frequency) του **CLOCK είναι ίση με 0.5Hz**. (Συμβουλευτείτε την παρουσίαση για τον simulator που βρίσκεται στα “Έγγραφα” του *eclass* του εργαστηρίου)
- Σε περίπτωση σχεδίασης/χρήσης υποκυκλώματος **πρέπει** να συμπεριλάβετε και ξεχωριστό screenshot για το υποκύκλωμα που χρησιμοποιείτε!!
- Η ακολουθία των ενεργειών που πρέπει να ακολουθήσετε είναι η εξής:
 - Reset
 - Ρύθμιση των καταχωρητών για *Παράλληλη Φόρτωση*
 - Παράλληλη Φόρτωση
 - Ρύθμιση των καταχωρητών για *Δεξιά Ολίσθηση*
 - Τέσσερις Δεξιές ολισθήσεις.

Το αποτέλεσμα της πρόσθεσης πρέπει να φαίνεται στον καταχωρητή A

ΚΥΚΛΩΜΑ

- Screenshot του κυκλώματος του σειριακού αθροιστή για $A=1010$, $B=0011$ όπου να φαίνεται το τελικό αποτέλεσμα σε LEDs/Output pins



Αναλυτική επεξήγηση της επιλογής σχεδίασης και της λειτουργίας του κυκλώματος

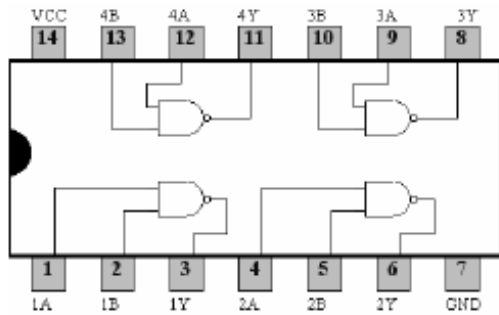
Όπως αναφέρεται και στο ερώτημα 5 χρησιμοποιούμε το 4 bit binary full adder για την σχεδίαση του κυκλώματος που τα συνδέουμε με το SHR για να κάνει δεξιά ολίσθηση. Συνδέουμε το Σ_1 στο SHR του πρώτου καταχωρητή ώστε να γίνεται σωστά η πρόθεση που θα χρησιμοποιήσουμε με την δεξιά ολίσθηση. Συνδέουμε την Σ_2 στο Data Input του flip flop ώστε να μεταφέρεται η άθροιση του. Όπως αναφέρεται και στην άσκηση συνδέουμε το Cin στο Q του flip flop για να έχουμε και το κρατούμενο. Επιπλέον, συνδέουμε τα LSB των καταχωρητών στον full adder που δεν χρειάζεται να συνδέσουμε κάποια άλλα bit εκτός του MSB του καταχωρητή A που το συνδέουμε στο A4 του αθροιστή για να μεταφέρεται το κρατούμενο.

Η λειτουργία του κυκλώματος είναι σωστή μετά την ακολουθία που αναφέρεται παραπάνω. Πρώτα παρατηρούμε ότι με την παράλληλη φόρτωση μεταδίδονται οι τιμές στους καταχωρητές και με την χρήση του SHR βλέπουμε ότι παίρνουμε το σωστό αποτέλεσμα μετά από τέσσερις δεξιές ολισθήσεις.

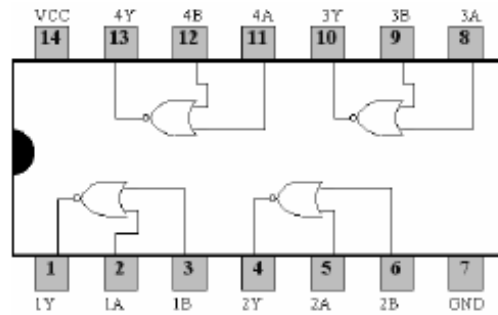
ΒΟΗΘΗΤΙΚΗ ΣΕΛΙΔΑ.

ΠΑΡΑΡΤΗΜΑ

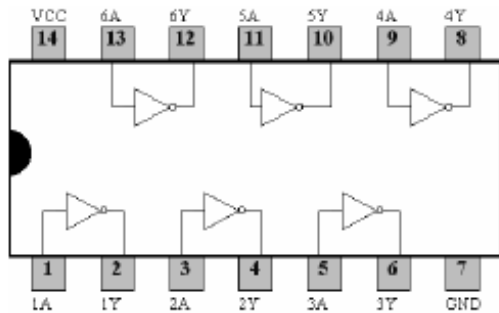
Διαγράμματα Ολοκληρωμένων



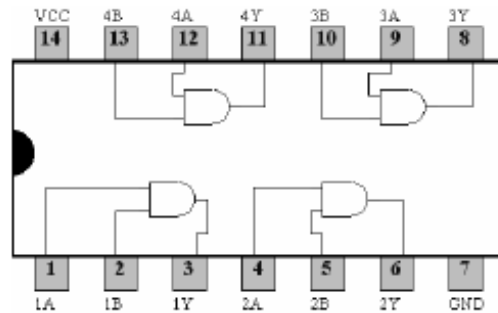
7400 Quad 2-Input NAND Gate



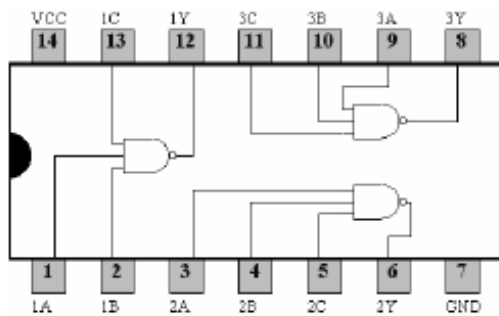
7402 Quad 2-Input NOR Gate



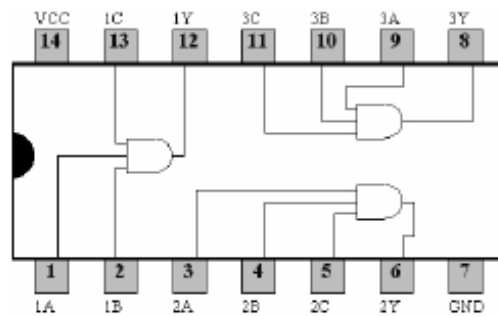
7404 Hex Inverter



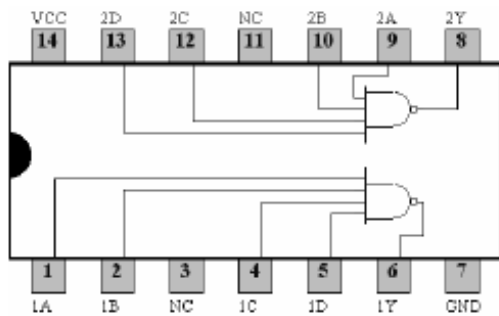
7408 Quad 2-Input AND Gate



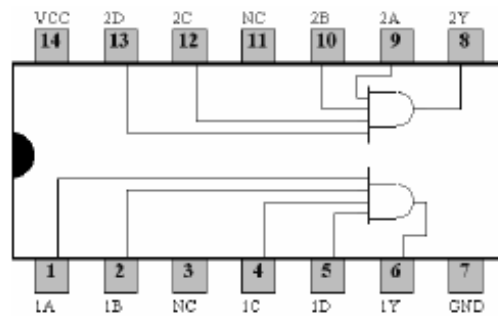
7410 Triple 3-Input NAND Gate



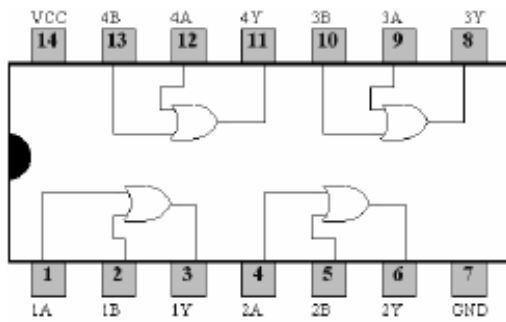
7411 Triple 3-Input AND Gate



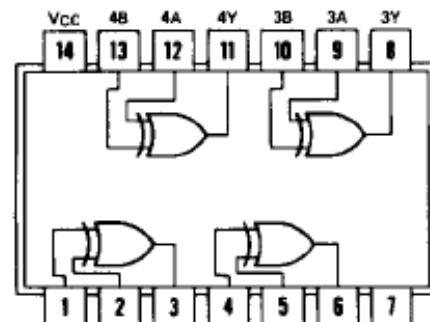
7420 Dual 4-Input NAND Gate



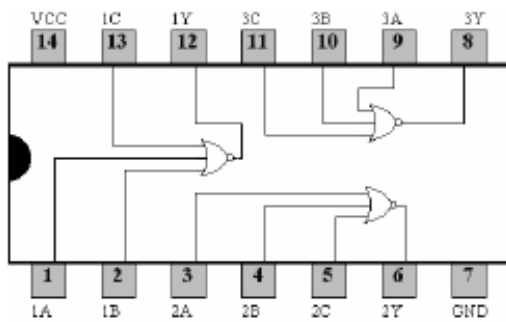
7421 Dual 4-Input AND Gate



7432 Quad 2-Input OR Gate



7486 Quad 2-Input XOR Gate

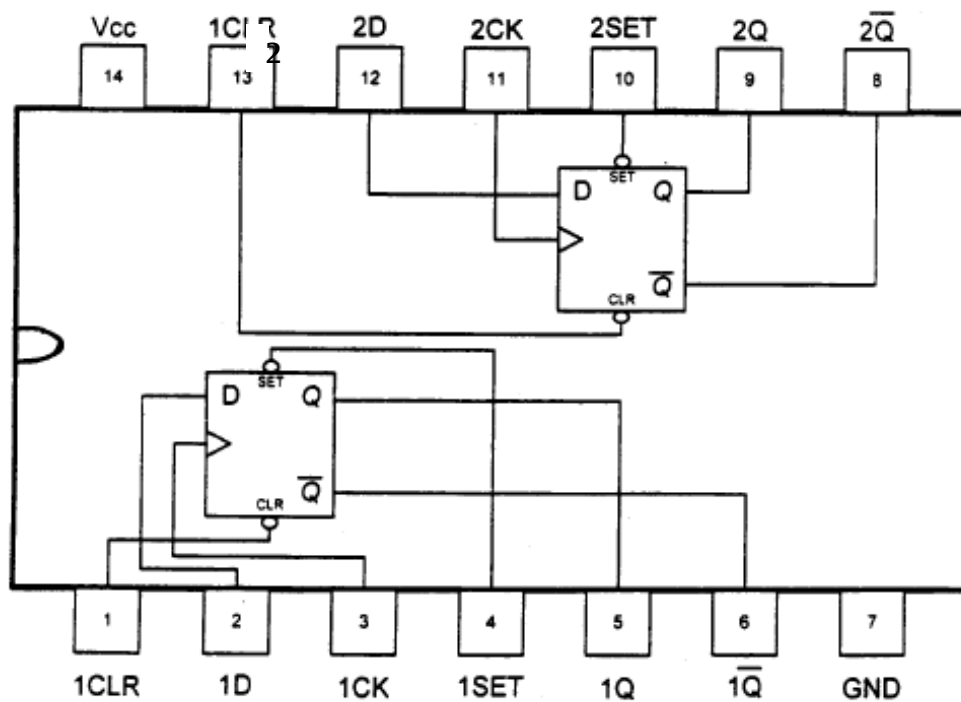


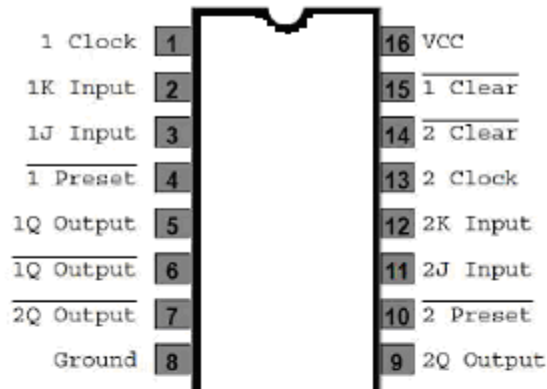
7427 Triple 3-Input NOR Gate

DUAL D-TYPE POSITIVE EDGE TRIGGERED FLIP-FLOPS WITH PRESET AND CLEAR

FUNCTION TABLE

INPUTS				OUTPUTS	
PRESET	CLEAR	CLOCK	D	Q	Q'
L	H	X	X	H	L
H	L	X	X	L	H
L	L	X	X	H	H
H	H	↑	H	H	L
H	H	↑	L	L	H
H	H	L	X	Q ₀	Q' ₀



DUAL JK FLIP_FLOP with CLEAR/PRESET**74112****Function Table**

Inputs					Outputs	
PR	CLR	CLK	J	K	Q	$\overline{Q}$
L	H	X	X	X	H	L
H	L	X	X	X	L	H
L	L	X	X	X	H (Note 1)	H (Note 1)
H	H	↓	L	L	Q_0	$\overline{Q}_0$
H	H	↓	H	L	H	L
H	H	↓	L	H	L	H
H	H	↓	H	H	Toggle	
H	H	H	X	X	Q_0	$\overline{Q}_0$

H = HIGH Logic Level

L = LOW Logic Level

X = Either LOW or HIGH Logic Level

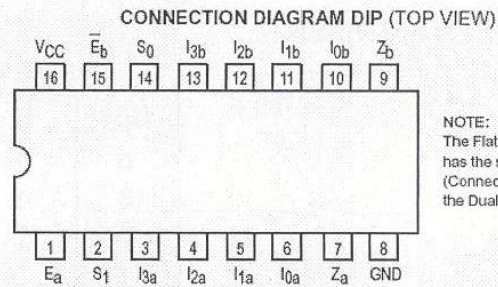
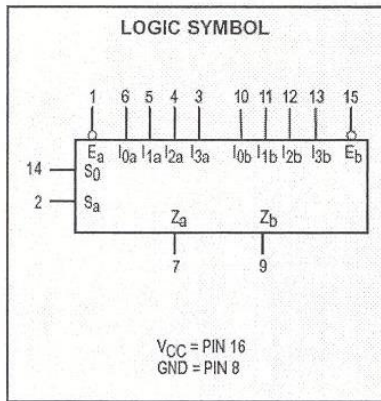
↓ = Negative Going Edge of Pulse

 Q_0 = The output logic level before the indicated input conditions were established.

Toggle = Each output changes to the complement of its previous level on each falling edge of the clock pulse.

Note 1: This configuration is nonstable; that is, it will not persist when preset and/or clear inputs return to their inactive (HIGH) level.

74153

DUAL 4-INPUT MULTIPLEXER

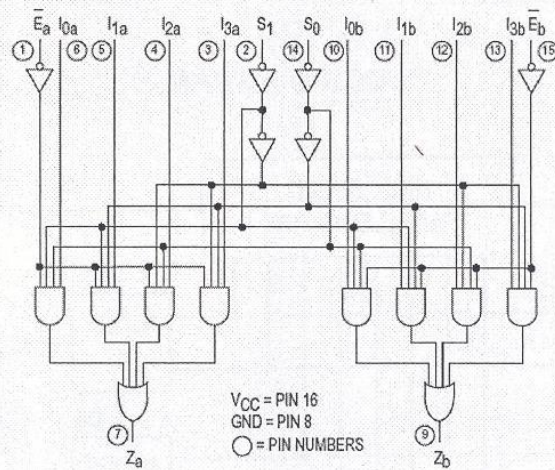
NOTE:
The Flatpak version
has the same pinouts
(Connection Diagram) as
the Dual In-Line Package.

PIN NAMES

S₀ Common Select Input
E Enable (Active LOW) Input
I₀, I₁ Multiplexer Inputs
Z Multiplexer Output (Note b)

LOADING (Note a)

HIGH	LOW
0.5 U.L.	0.25 U.L.
0.5 U.L.	0.25 U.L.
0.5 U.L.	0.25 U.L.
10 U.L.	5 (2.5) U.L.

LOGIC DIAGRAM**TRUTH TABLE**

SELECT INPUTS		INPUTS (a or b)					OUTPUT
S ₀	S ₁	E	I ₀	I ₁	I ₂	I ₃	Z
X	X	H	X	X	X	X	L
L	L	L	L	X	X	X	L
L	L	L	H	X	X	X	H
H	L	L	X	L	X	X	L
H	L	L	X	H	X	X	H
L	H	L	X	X	L	X	L
L	H	L	X	X	H	X	H
H	H	L	X	X	X	L	L
H	H	L	X	X	X	H	H

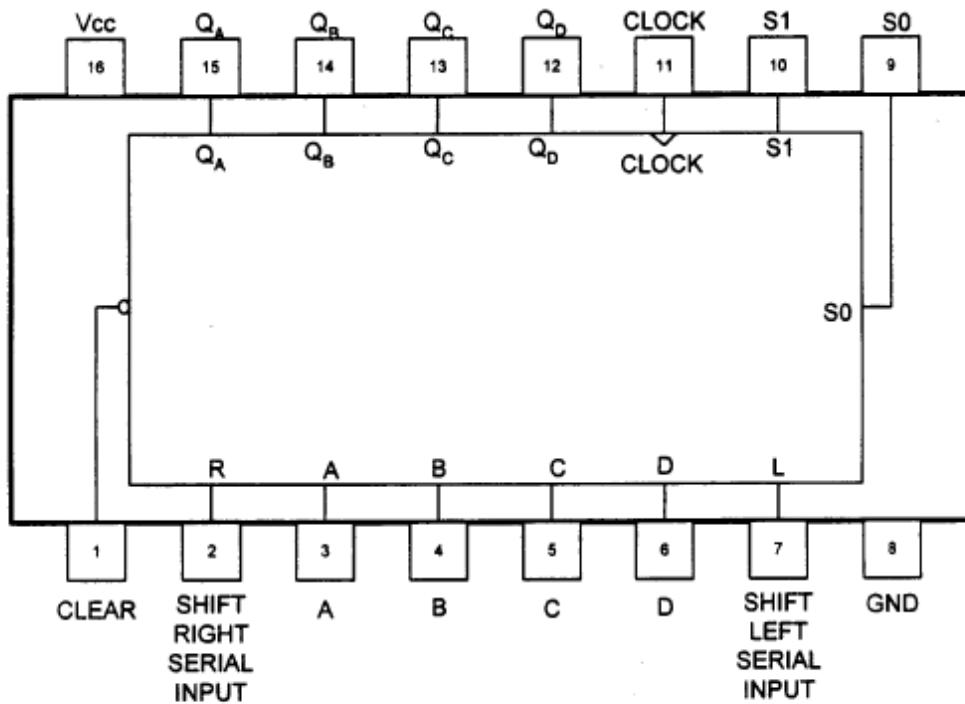
H = HIGH Voltage Level
L = LOW Voltage Level
X = Don't Care

74194

4 - BIT BIDIRECTIONAL UNIVERSAL SHIFT REGISTER

FUNCTION TABLE

INPUTS										OUTPUTS			
CLEAR	MODE		CLOCK	SERIAL		PARALLEL				Q _A	Q _B	Q _C	Q _D
	S ₁	S ₀		LEFT	RIGHT	A	B	C	D				
L	X	X	X	X	X	X	$\bar{E}_a$	X	X	L	L	L	L
H	X	X	L	X	X	X	X	X	X	Q _{A0}	Q _{B0}	Q _{C0}	Q _{D0}
H	H	H	↑	X	X	a	b	c	d	a	b	c	d
H	L	H	↑	X	H	X	X	X	X	H	Q _{An}	Q _{Bn}	Q _{Cn}
H	L	H	↑	X	L	X	X	X	X	L	Q _{An}	Q _{Bn}	Q _{Cn}
H	H	L	↑	H	X	X	X	X	X	Q _{Bn}	Q _{Cn}	Q _{Dn}	H
H	H	L	↑	L	X	X	X	X	X	Q _{Bn}	Q _{Cn}	Q _{Dn}	L
H	L	L	X	X	X	X	X	X	X	Q _{A0}	Q _{B0}	Q _{C0}	Q _{D0}





August 1986
Revised March 2000

DM74LS283

4-Bit Binary Adder with Fast Carry

General Description

These full adders perform the addition of two 4-bit binary numbers. The sum (Σ) outputs are provided for each bit and the resultant carry (C4) is obtained from the fourth bit. These adders feature full internal look ahead across all four bits. This provides the system designer with partial look-ahead performance at the economy and reduced package count of a ripple-carry implementation.

The adder logic, including the carry, is implemented in its true form meaning that the end-around carry can be accomplished without the need for logic or level inversion.

Features

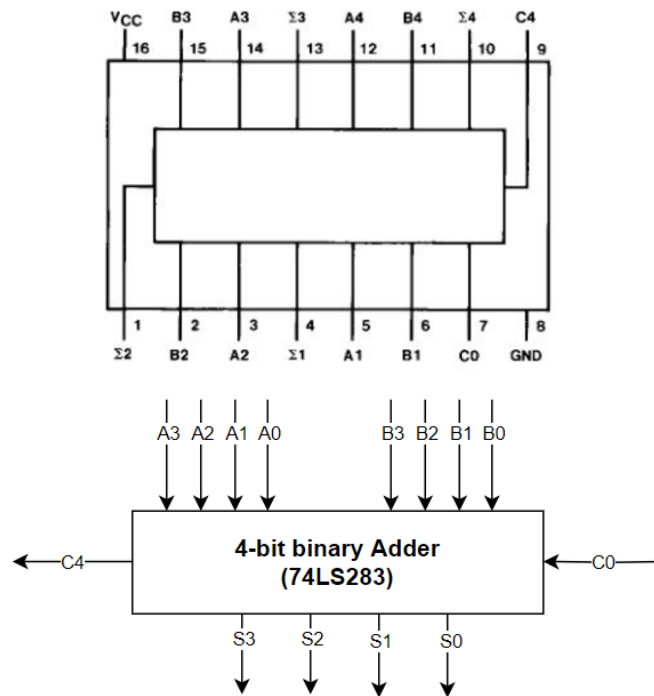
- Full-carry look-ahead across the four bits
- Systems achieve partial look-ahead performance with the economy of ripple carry
- Typical add times
 - Two 8-bit words 25 ns
 - Two 16-bit words 45 ns
- Typical power dissipation per 4-bit adder 95 mW

Ordering Code:

Order Number	Package Number	Package Description
DM74LS283M	M16A	16-Lead Small Outline Integrated Circuit (SOIC), J EDEC MS-012, 0.150 Narrow
DM74LS283N	N16E	16-Lead Plastic Dual-In-Line Package (PDIP), J EDEC MS-001, 0.300 Wide

Devices also available in Tape and Reel. Specify by appending the suffix letter "X" to the ordering code.

Connection Diagram



DM74LS283 4-BIT BINARY ADDER WITH FAST CARRY

74LS181 – 4-bit ARITHMETIC LOGIC UNIT

TABLE 2

SELECTION				ACTIVE-HIGH DATA		
				M = H LOGIC FUNCTIONS	M = L; ARITHMETIC OPERATIONS	
S3	S2	S1	S0		$\overline{C}_n = H$ (no carry)	$\overline{C}_n = L$ (with carry)
L	L	L	L	$F = \overline{A}$	$F = A$	$F = A \text{ PLUS } 1$
L	L	L	H	$F = \overline{A + B}$	$F = A + B$	$F = (A + B) \text{ PLUS } 1$
L	L	H	L	$F = \overline{AB}$	$F = A + \overline{B}$	$F = (A + \overline{B}) \text{ PLUS } 1$
L	L	H	H	$F = 0$	$F = \text{MINUS } 1 \text{ (2's COMPL)}$	$F = \text{ZERO}$
L	H	L	L	$F = \overline{AB}$	$F = A \text{ PLUS } \overline{AB}$	$F = A \text{ PLUS } \overline{AB} \text{ PLUS } 1$
L	H	L	H	$F = \overline{B}$	$F = (A + B) \text{ PLUS } \overline{AB}$	$F = (A + B) \text{ PLUS } \overline{AB} \text{ PLUS } 1$
L	H	H	L	$F = A \oplus B$	$F = A \text{ MINUS } B \text{ MINUS } 1$	$F = A \text{ MINUS } B$
L	H	H	H	$F = \overline{AB}$	$F = \overline{AB} \text{ MINUS } 1$	$F = \overline{AB}$
H	L	L	L	$F = \overline{A + B}$	$F = A \text{ PLUS } AB$	$F = A \text{ PLUS } AB \text{ PLUS } 1$
H	L	L	H	$F = A \oplus B$	$F = A \text{ PLUS } B$	$F = A \text{ PLUS } B \text{ PLUS } 1$
H	L	H	L	$F = B$	$F = (A + \overline{B}) \text{ PLUS } AB$	$F = (A + \overline{B}) \text{ PLUS } AB \text{ PLUS } 1$
H	L	H	H	$F = AB$	$F = AB \text{ MINUS } 1$	$F = AB$
H	H	L	L	$F = 1$	$F = A \text{ PLUS } A^\dagger$	$F = A \text{ PLUS } A \text{ PLUS } 1$
H	H	L	H	$F = A + \overline{B}$	$F = (A + B) \text{ PLUS } A$	$F = (A + B) \text{ PLUS } A \text{ PLUS } 1$
H	H	H	L	$F = A + B$	$F = (A + \overline{B}) \text{ PLUS } A$	$F = (A + \overline{B}) \text{ PLUS } A \text{ PLUS } 1$
H	H	H	H	$F = A$	$F = A \text{ MINUS } 1$	$F = A$

[†] Each bit is shifted to the next more significant position.

